

Database Design

Screenshot of the database connection

```
kristin_w_dev@cloudshell:~ (su26-cs411-500600)$ gcloud sql connect db-su26-cs411 --user=Kristin
Starting Cloud SQL Proxy: [/usr/bin/cloud-sql-proxy su26-cs411-500600:us-central1:db-su26-cs411 --port 9470]
2026/06/30 16:59:12 Authorizing with Application Default Credentials
2026/06/30 16:59:12 [su26-cs411-500600:us-central1:db-su26-cs411] Listening on 127.0.0.1:9470
2026/06/30 16:59:12 The proxy has started successfully and is ready for new connections!
Connecting to database with SQL user [Kristin].Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 180165
Server version: 8.0.44-google (Google)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

Data Definition Language (DDL) commands

SQL

```
CREATE TABLE Users(
    UserID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(250) DEFAULT NULL,
    Email VARCHAR(250) NOT NULL UNIQUE);
```

SQL

```
CREATE TABLE CelestialObject(
    ObjectID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(250),
    Magnitude FLOAT,
    ObjectCategory VARCHAR(250),
    RightAscension FLOAT,
    Declination FLOAT,
    Constellation VARCHAR(250));
```

SQL

```
CREATE TABLE LightPollutionObservation(  
    ObservationID INT AUTO_INCREMENT PRIMARY KEY,  
    Latitude FLOAT,  
    Longitude FLOAT,  
    LimitingMag FLOAT);
```

SQL

```
CREATE TABLE CommunityReport(  
    ReportID INT AUTO_INCREMENT PRIMARY KEY,  
    UserID INT,  
    Latitude FLOAT,  
    Longitude FLOAT,  
    ReportText TEXT,  
    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY(UserID) REFERENCES Users(UserID)  
    ON DELETE SET NULL);
```

SQL

```
CREATE TABLE ObservationList(  
    ListID INT AUTO_INCREMENT PRIMARY KEY,  
    UserID INT NOT NULL,  
    Latitude FLOAT,  
    Longitude FLOAT,  
    ListName VARCHAR(250),  
    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY(UserID) REFERENCES Users(UserID)  
    ON DELETE CASCADE);
```

SQL

```
CREATE TABLE SavedObject(  
    ListID INT NOT NULL,  
    ObjectID INT NOT NULL,  
    ObservedStatus VARCHAR(250),  
    AddedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

PRIMARY KEY(ListID, ObjectID),
FOREIGN KEY(ListID) REFERENCES ObservationList(ListID)
ON DELETE CASCADE,
FOREIGN KEY(ObjectID) REFERENCES CelestialObject(ObjectID)
ON DELETE CASCADE);

```

Count queries showing at least 1000 rows in 3 different tables

```

mysql> SELECT COUNT(*) FROM CelestialObject;
+-----+
| COUNT(*) |
+-----+
| 132852 |
+-----+
1 row in set (0.97 sec)

mysql> SELECT COUNT(*) FROM LightPollutionObservation;
+-----+
| COUNT(*) |
+-----+
| 12525 |
+-----+
1 row in set (0.16 sec)

mysql> SELECT COUNT(*) FROM Users;
+-----+
| COUNT(*) |
+-----+
| 2000 |
+-----+
1 row in set (0.16 sec)

```

* The Users table has AI-generated data (names and email addresses).

```
Database changed
mysql> SELECT COUNT(*) FROM SavedObject
-> ;
+-----+
| COUNT(*) |
+-----+
| 134263 |
+-----+
1 row in set (0.45 sec)
```

* SavedObject is loaded with generated data for the ObservedStatus and AddedAt attributes, courtesy of Claude.

Top 15 rows of advanced queries

Query 1: constellation visibility

In the CelestialObject table, the constellation attribute refers to the general area of the sky a star is in, not membership in a constellation. First, this query finds the number of stars actually belonging to a constellation by counting the number of significantly bright stars (magnitude < 3) in the general area of the sky. The second CTE (VisibleStars) uses the user's provided latitude and longitude to count the number of stars per constellation that are brighter than the user's light pollution threshold. Finally, the query outputs a table where each row contains the constellation's name as a 3 letter code, the number of stars the user can see from this constellation, the total number of stars in the constellation, and visibility of the constellation as the percentage of total stars visible to the user. The results from this query depend on providing a latitude and longitude. Champaign, Illinois' latitude and longitude was used for the screenshot below (latitude = 40.1164°, longitude = -88.2434°).

```
SQL
WITH TotalStars AS (
  -- Table 1: Counts every significant star in each constellation
  SELECT Constellation,
         COUNT(ObjectID) AS StarCount
  FROM CelestialObject
  WHERE Constellation IS NOT NULL
        AND Constellation != ''
        AND Magnitude < 3      -- Only the bright 'connect the dots' stars
  GROUP BY Constellation
),
VisibleStars AS (
  -- Table 2: Only count the stars brighter than the local light pollution
```

```

SELECT Constellation,
       COUNT(ObjectID) AS StarsVisible
FROM CelestialObject
WHERE Constellation IS NOT NULL
      AND Constellation != ''
      AND Magnitude < 3          -- Only the bright 'connect the dots' stars
      AND Magnitude <= (
        -- Find the closest local average limiting magnitude
        -- If NULL, defaults to suburban/rural average of 6
        SELECT COALESCE(AVG(LimitingMag), 6)
        FROM (
          -- Limit calculation to the 3 closest light pollution observations
          SELECT LimitingMag
          FROM LightPollutionObservation
          -- Champaign, IL: latitude = 40.1164, long = -88.2434
          WHERE Latitude BETWEEN 40.1164 - 0.1448 AND 40.1164 + 0.1448
                AND Longitude BETWEEN
                  -88.2434 - 10 / 69.17 * COS(RADIANS(40.1164))
                  AND -88.2434 + 10 / 69.17 * COS(RADIANS(40.1164))
          ORDER BY ABS(40.1164 - Latitude),
                  ABS(-88.2434 - Longitude)
          LIMIT 3
        ) AS ThreeClosest
      )
GROUP BY Constellation
)

-- Join and format constellation visibility as a percentage
SELECT
  t.Constellation,
  COALESCE(StarsVisible, 0) AS VisibleCount,
  StarCount,
  CONCAT(ROUND((COALESCE(StarsVisible, 0) / StarCount) * 100, 0), '%')
  AS VisibilityPercentage
FROM TotalStars t
  LEFT JOIN VisibleStars USING (Constellation)
ORDER BY (VisibleCount / StarCount) DESC,
         StarCount DESC;

```

```

mysql> WITH TotalStars AS (
->   -- Table 1: Counts every significant star in each constellation
->   SELECT Constellation,
->         COUNT(ObjectID) AS StarCount
->   FROM CelestialObject
->   WHERE Constellation IS NOT NULL
->         AND Constellation != ''
->         AND Magnitude < 3      -- Only the bright 'connect the dots' stars
->   GROUP BY Constellation
-> ),
-> VisibleStars AS (
->   -- Table 2: Only count the stars brighter than the local light pollution
->   SELECT Constellation,
->         COUNT(ObjectID) AS StarsVisible
->   FROM CelestialObject
->   WHERE Constellation IS NOT NULL
->         AND Constellation != ''
->         AND Magnitude < 3      -- Only the bright 'connect the dots' stars
->         AND Magnitude <= (
->           -- Find the closest local average limiting magnitude
->           -- If NULL, defaults to suburban/rural average of 6
->           SELECT COALESCE(AVG(LimitingMag), 6)
->           FROM (
->             -- Limit calculation to the 3 closest light pollution observations
->             SELECT LimitingMag
->             FROM LightPollutionObservation
->             -- Champaign, IL: latitude = 40.1164, long = -88.2434
->             WHERE Latitude BETWEEN 40.1164 - 0.1448 AND 40.1164 + 0.1448
->                   AND Longitude BETWEEN
->                   -88.2434 - 10 / 69.17 * COS(RADIANS(40.1164))
->                   AND -88.2434 + 10 / 69.17 * COS(RADIANS(40.1164))
->             ORDER BY ABS(40.1164 - Latitude),
->                   ABS(-88.2434 - Longitude)
->             LIMIT 3
->           ) AS ThreeClosest
->         )
->   GROUP BY Constellation
-> )
-> -- Join and format constellation visibility as a percentage
-> SELECT
->   t.Constellation,
->   COALESCE(StarsVisible, 0) AS VisibleCount,
->   StarCount,
->   CONCAT(ROUND((COALESCE(StarsVisible, 0) / StarCount) * 100, 0), '%')
->   AS VisibilityPercentage
-> FROM TotalStars t
->   LEFT JOIN VisibleStars USING (Constellation)
-> ORDER BY (VisibleCount / StarCount) DESC,
->          StarCount DESC
-> LIMIT 15;

```

Constellation	VisibleCount	StarCount	VisibilityPercentage
Pav	1	1	100%
Lyr	1	1	100%
PsA	1	1	100%
CMa	4	5	80%
Cru	3	4	75%
Ori	5	8	63%
Aur	3	5	60%
Gem	3	5	60%
Car	3	6	50%
UMa	3	6	50%
Tau	2	4	50%
Hya	1	2	50%
Gru	1	2	50%
UMi	1	2	50%
CMi	1	2	50%

15 rows in set (0.18 sec)

Query 2: observation list metadata

Query 2 provides summary information about the items on one ObservationList. It groups all of the items on an ObservationList by the corresponding CelestialObject's ObjectCategory, and counts the total items from that category, the number of observed items from that category, and a completion rate for the category. Even though ListID 2 contains a SavedObject corresponding with every row of CelestialObject (over 100,000 rows), the output below has less than 15 rows. This is for a few reasons: there are only 19 ObjectCategories in the CelestialObject table, some Object Categories have very few associated CelestialObjects (so completion is easier to reach), and the query intentionally filters out Object Categories that a user has completed.

SQL

```
SELECT ObjectCategory,
       COUNT(ObjectID) AS TotalSaved,
       SUM(ObservedStatus = 'Seen') AS Observed,
       ROUND(SUM(ObservedStatus = 'Seen') / COUNT(ObjectID) * 100, 0)
         AS CompletionRate
FROM ObservationList
   NATURAL JOIN SavedObject
   NATURAL JOIN CelestialObject
WHERE ListID = 2
GROUP BY ObjectCategory
HAVING CompletionRate < 100
ORDER BY CompletionRate DESC;
```

-- provide list id here for query test

```
mysql> SELECT ObjectCategory,
->         COUNT(ObjectID) AS TotalSaved,
->         SUM(ObservedStatus = 'Seen') AS Observed,
->         ROUND(SUM(ObservedStatus = 'Seen') / COUNT(ObjectID) * 100, 0)
->         AS CompletionRate
-> FROM ObservationList
->     NATURAL JOIN SavedObject
->     NATURAL JOIN CelestialObject
-> WHERE ListID = 2 -- provide list id here for query test
-> GROUP BY ObjectCategory
-> HAVING CompletionRate < 100
-> ORDER BY CompletionRate DESC;
```

ObjectCategory	TotalSaved	Observed	CompletionRate
DS	91	90	99
Nb	201	197	98
TS	32	31	97
C+N	136	94	69
OC	590	265	45
Gb	153	30	20
Pl	134	27	20
Gx	8682	1559	18
Star	119625	1637	1

9 rows in set (2.16 sec)

Query 3: community reports near a given location

Query 3 takes a latitude and longitude acquired from the user's location and returns information about CommunityReports and LightPollutionObservations within 10 miles. In this example query, a latitude and longitude corresponding to Chicago, Illinois was used.

SQL

```
-- Champaign: 40.116 x -88.2434
-- Chicago: 41.8832 x -87.6324

SELECT ReportText,
       CreatedAt,
       DATEDIFF(NOW(), CreatedAt) AS DaysAgo,
       COALESCE(AVG(LimitingMag), 6) AS AvgLimitingMag,
       COUNT(ObservationID) AS NearbyObservations
FROM CommunityReport c
-- Combine all CommunityReports with any LightPollutionObservations within 10
miles
LEFT JOIN LightPollutionObservation l
-- 10 land miles = (approx) 0.1448 degrees latitude shift
ON l.Latitude BETWEEN c.Latitude - 0.1448 AND c.Latitude + 0.1448
-- 10 land miles = longitude shift scaled by latitude for Earth's
curvature
```

```

        AND l.Longitude BETWEEN
            c.Longitude - 10 / 69.17 * COS(RADIANS(c.Latitude))
            AND c.Longitude + 10 / 69.17 * COS(RADIANS(c.Latitude))
-- Filter only CommunityReports within 10 miles of user (assumed to be in Chicago)
WHERE c.Latitude BETWEEN 41.8832 - 0.1448 AND 41.8832 + 0.1448
    AND c.Longitude BETWEEN
        -87.6324 - 10 / 69.17 * COS(RADIANS(41.8832))
        AND -87.6324 + 10 / 69.17 * COS(RADIANS(41.8832))
GROUP BY ReportID,
        ReportText,
        CreatedAt
ORDER BY DaysAgo;

```

Since the ReportText attribute allows a fairly large amount of text, the top 15 rows are displayed with the rows and columns flipped:

```

mysql> SELECT ReportText,
->     CreatedAt,
->     DATEDIFF(NOW(), CreatedAt) AS DaysAgo,
->     COALESCE(AVG(LimitingMag), 6) AS AvgLimitingMag,
->     COUNT(ObservationID) AS NearbyObservations
-> FROM CommunityReport c
-> -- Combine all CommunityReports with any LightPollutionObservations within 10 miles
-> LEFT JOIN LightPollutionObservation l
->     -- 10 land miles = (approx) 0.1448 degrees latitude shift
->     ON l.Latitude BETWEEN c.Latitude - 0.1448 AND c.Latitude + 0.1448
->     -- 10 land miles = longitude shift scaled by latitude for Earth's curvature
->     AND l.Longitude BETWEEN
->         c.Longitude - 10 / 69.17 * COS(RADIANS(c.Latitude))
->         AND c.Longitude + 10 / 69.17 * COS(RADIANS(c.Latitude))
-> -- Filter only CommunityReports within 10 miles of user (assumed to be in Chicago)
-> WHERE c.Latitude BETWEEN 41.8832 - 0.1448 AND 41.8832 + 0.1448
->     AND c.Longitude BETWEEN
->         -87.6324 - 10 / 69.17 * COS(RADIANS(41.8832))
->         AND -87.6324 + 10 / 69.17 * COS(RADIANS(41.8832))
-> GROUP BY ReportID,
->     ReportText,
->     CreatedAt
-> ORDER BY DaysAgo
-> LIMIT 15 \G
***** 1. row *****
        ReportText: Rogers Park, northern edge of the city along the lake. Loyola campus
lighting is a factor but stepping onto the beach at night and looking out
over the water gave the darkest sky of anywhere on this list. Still urban,
but the most usable of the group.
        CreatedAt: 2026-06-16 22:30:00
        DaysAgo: 18
        AvgLimitingMag: 3
NearbyObservations: 3
***** 2. row *****
        ReportText: Hyde Park, near the University of Chicago campus. Promontory Point on the
lakefront gave the clearest eastward view of the whole cluster - could
pick out a few more stars than downtown, but still far from anything
resembling dark.
        CreatedAt: 2026-04-30 22:00:00
        DaysAgo: 65
        AvgLimitingMag: 3
NearbyObservations: 2

```

```

***** 3. row *****
    ReportText: Logan Square -- the boulevard mature tree canopy blocks a lot of direct
streetlight glare at ground level, which helps more than expected.
Still did not get below mag 2 even on a clear, low-humidity night.
    CreatedAt: 2026-04-18 21:15:00
    DaysAgo: 77
    AvgLimitingMag: 2
NearbyObservations: 3
***** 4. row *****
    ReportText: Bronzeville lakefront - quieter streets than downtown but still well within
the general Chicago light dome. Promontory-adjacent areas nearby are worth
checking if you want a slightly better lakefront view.
    CreatedAt: 2026-04-05 20:45:00
    DaysAgo: 90
    AvgLimitingMag: 2
NearbyObservations: 3
***** 5. row *****
    ReportText: Garfield Park, west side, near the conservatory. A bit further from the
core than most of my other reports and it shows - slightly darker overhead
sky, though the horizon in every direction still glows from surrounding
neighborhoods.
    CreatedAt: 2026-03-24 21:00:00
    DaysAgo: 102
    AvgLimitingMag: 2.5
NearbyObservations: 2
***** 6. row *****
    ReportText: Lakeview near Wrigley Field - on non-game nights it is comparable to the
rest of the north side, but stadium lighting during a night game lights up
the whole neighborhood. Plan around the Cubs schedule.
    CreatedAt: 2026-03-12 21:45:00
    DaysAgo: 114
    AvgLimitingMag: 2
NearbyObservations: 3
***** 7. row *****
    ReportText: McKinley Park sits right next to the Stevenson Expressway - the highway
lighting alone ruins the southern horizon. Northern sky away from the
interchange was a bit more usable.
    CreatedAt: 2026-02-27 20:30:00
    DaysAgo: 127
    AvgLimitingMag: 2
NearbyObservations: 3

```

```

***** 8. row *****
ReportText: Humboldt Park boathouse area, away from the main road, was one of the
better west side spots. Enough distance from streetlights that the belt
of Orion was easy to trace. Still solidly urban-grade sky overall.
CreatedAt: 2026-02-15 21:00:00
DaysAgo: 139
AvgLimitingMag: 2
NearbyObservations: 3
***** 9. row *****
ReportText: Bucktown is basically an extension of Wicker Park in terms of light
pollution. New construction along Damen Ave keeps adding more glow every
season. Not a serious observing spot but fine for a casual look at the
Moon.
CreatedAt: 2026-02-03 22:15:00
DaysAgo: 151
AvgLimitingMag: 2
NearbyObservations: 3
***** 10. row *****
ReportText: Bridgeport — industrial corridor along the river adds sodium-vapor orange
glow. On Sox home game nights the stadium lighting is visible for miles and
makes things noticeably worse. Check the schedule before you go.
CreatedAt: 2026-01-21 21:30:00
DaysAgo: 164
AvgLimitingMag: 2
NearbyObservations: 3
***** 11. row *****
ReportText: Douglas Park on the west side has decent tree cover that blocks direct
sightlines to a lot of the surrounding streetlights. Stood near the lagoon
and could make out the Big Dipper clearly, which says more about the rest
of the city than this park.
CreatedAt: 2026-01-09 21:00:00
DaysAgo: 176
AvgLimitingMag: 2
NearbyObservations: 3
***** 12. row *****
ReportText: Wicker Park bar district lighting on Milwaukee Ave is brutal after dark.
Residential side streets a few blocks off the main strip are slightly
calmer but the overall dome from downtown still dominates the sky.
CreatedAt: 2025-12-27 23:00:00
DaysAgo: 189
AvgLimitingMag: 2
NearbyObservations: 3

***** 13. row *****
ReportText: Lincoln Park — the lakefront running path away from the zoo lighting was
the best spot I found. Looking out over the water at night you can pick up
a handful of first-magnitude stars. Still nowhere close to suburban skies.
CreatedAt: 2025-12-10 22:00:00
DaysAgo: 206
AvgLimitingMag: 2
NearbyObservations: 3
***** 14. row *****
ReportText: Pilsen has some darker corners near the old rail yards south of 18th St
where streetlighting is sparse. Still heavily washed out overall but a
noticeable step down in glow compared to the Loop just a couple miles
north.
CreatedAt: 2025-11-29 21:45:00
DaysAgo: 217
AvgLimitingMag: 2
NearbyObservations: 3
***** 15. row *****
ReportText: Chinatown — decorative lantern lighting along Wentworth Ave adds a layer of
local glow on top of the general city dome. Small pocket park near the
library was marginally darker than the main strip.
CreatedAt: 2025-11-16 20:30:00
DaysAgo: 230
AvgLimitingMag: 2
NearbyObservations: 3
15 rows in set (0.07 sec)

```

Just in case flipping the rows into columns isn't allowed:

ReportText	CreatedAt	DaysAgo	AvgLimitingMag	NearbyObservations
+-----+-----+-----+-----+-----+				
Rogers Park, northern edge of the city along the lake. Loyola campus lighting is a factor but stepping onto the beach at night and looking out over the water gave the darkest sky of anywhere on this list. Still urban, but the most usable of the group. 2026-06-16 22:30:00 18	3	3		
Hyde Park, near the University of Chicago campus. Promontory Point on the lakefront gave the clearest eastward view of the whole cluster - could pick out a few more stars than downtown, but still far from anything resembling dark. 2026-04-30 22:00:00 65	3	2		
Logan Square -- the boulevard mature tree canopy blocks a lot of direct streetlight glare at ground level, which helps more than expected. Still did not get below mag 2 even on a clear, low-humidity night. 2026-04-18 21:15:00 77	2	3		
Bronzeville lakefront - quieter streets than downtown but still well within the general Chicago light dome. Promontory-adjacent areas nearby are worth checking if you want a slightly better lakefront view. 2026-04-05 20:45:00 90	2	3		
Garfield Park, west side, near the conservatory. A bit further from the core than most of my other reports and it shows - slightly darker overhead sky, though the horizon in every direction still glows from surrounding neighborhoods. 2026-03-24 21:00:00 102	2.5	2		
Lakeview near Wrigley Field - on non-game nights it is comparable to the rest of the north side, but stadium lighting during a night game lights up the whole neighborhood. Plan around the Cubs schedule. 2026-03-12 21:45:00 114	2	3		
McKinley Park sits right next to the Stevenson Expressway - the highway lighting alone ruins the southern horizon. Northern sky away from the interchange was a bit more usable. 2026-02-27 20:30:00 127	2	3		
Humboldt Park boathouse area, away from the main road, was one of the better west side spots. Enough distance from streetlights that the belt of Orion was easy to trace. Still solidly urban-grade sky overall. 2026-02-15 21:00:00 139	2	3		
Bucktown is basically an extension of Wicker Park in terms of light pollution. New construction along Damen Ave keeps adding more glow every season. Not a serious observing spot but fine for a casual look at the Moon. 2026-02-03 22:15:00 151	2	3		
Bridgeport - industrial corridor along the river adds sodium-vapor orange glow. On Sox home game nights the stadium lighting is visible for miles and makes things noticeably worse. Check the schedule before you go. 2026-01-21 21:30:00 164	2	3		
Douglas Park on the west side has decent tree cover that blocks direct sightlines to a lot of the surrounding streetlights. Stood near the lagoon and could make out the Big Dipper clearly, which says more about the rest of the city than this park. 2026-01-09 21:00:00 176	2	3		
Wicker Park bar district lighting on Milwaukee Ave is brutal after dark. Residential side streets a few blocks off the main strip are slightly calmer but the overall dome from downtown still dominates the sky. 2025-12-27 23:00:00 189	2	3		
Lincoln Park - the lakefront running path away from the zoo lighting was the best spot I found. Looking out over the water at night you can pick up a handful of first-magnitude stars. Still nowhere close to suburban skies. 2025-12-10 22:00:00 206	2	3		
+-----+-----+-----+-----+-----+				
Pilsen has some darker corners near the old rail yards south of 18th St where streetlighting is sparse. Still heavily washed out overall but a noticeable step down in glow compared to the Loop just a couple miles north. 2025-11-29 21:45:00 217	2	3		
Chinatown - decorative lantern lighting along Wentworth Ave adds a layer of local glow on top of the general city dome. Small pocket park near the library was marginally darker than the main strip. 2025-11-16 20:30:00 230	2	3		
+-----+-----+-----+-----+-----+				
15 rows in set (0.09 sec)				

Advanced query performance prior to indexing

Query 1

```
mysql> EXPLAIN ANALYZE WITH TotalStars AS (  
-> -- Table 1: Counts every significant star in each constellation  
-> SELECT Constellation,  
->        COUNT(ObjectID) AS StarCount  
-> FROM CelestialObject  
-> WHERE Constellation IS NOT NULL  
->        AND Constellation != ''  
->        AND Magnitude < 3          -- Only the bright 'connect the dots' stars  
-> GROUP BY Constellation  
-> ),  
-> VisibleStars AS (  
-> -- Table 2: Only count the stars brighter than the local light pollution  
-> SELECT Constellation,  
->        COUNT(ObjectID) AS StarsVisible  
-> FROM CelestialObject  
-> WHERE Constellation IS NOT NULL  
->        AND Constellation != ''  
->        AND Magnitude < 3          -- Only the bright 'connect the dots' stars  
->        AND Magnitude <= (  
->        -- Find the closest local average limiting magnitude  
->        -- If NULL, defaults to suburban/rural average of 6  
->        SELECT COALESCE(AVG(LimitingMag), 6)  
->        FROM (  
->        -- Limit calculation to the 3 closest light pollution observations  
->        SELECT LimitingMag  
->        FROM LightPollutionObservation  
->        -- Champaign, IL: latitude = 40.1164, long = -88.2434  
->        WHERE Latitude BETWEEN 40.1164 - 0.1448 AND 40.1164 + 0.1448  
->        AND Longitude BETWEEN  
->        -88.2434 - 10 / 69.17 * COS(RADIANS(40.1164))  
->        AND -88.2434 + 10 / 69.17 * COS(RADIANS(40.1164))  
->        )  
->        )  
-> )  
-> SELECT Constellation, StarCount, StarsVisible  
-> FROM TotalStars, VisibleStars  
-> )
```

```

->          AND -88.2434 + 10 / 69.17 * COS(RADIANS(40.1164))
->      ORDER BY ABS(40.1164 - Latitude),
->          ABS(-88.2434 - Longitude)
->      LIMIT 3
->  ) AS ThreeClosest
->  )
-> GROUP BY Constellation
-> )
->
-> -- Join and format constellation visibility as a percentage
-> SELECT
->     t.Constellation,
->     COALESCE(StarsVisible, 0) AS VisibleCount,
->     StarCount,
->     CONCAT(ROUND((COALESCE(StarsVisible, 0) / StarCount) * 100, 0), '%')
->         AS VisibilityPercentage
-> FROM TotalStars t
->     LEFT JOIN VisibleStars USING (Constellation)
-> ORDER BY (VisibleCount / StarCount) DESC,
->     StarCount DESC;

```

```

-----+
| -> Sort: `(VisibleCount / t.StarCount)` DESC, t.StarCount DESC (actual time=22.7..22.7 rows=52 loops=1)
|   -> Stream results (cost=441 rows=0) (actual time=22.5..22.6 rows=52 loops=1)
|     -> Left hash join (<hash>(VisibleStars.Constellation)=<hash>(t.Constellation)), extra conditions: (VisibleStars.Constellation = t.Constellation) (
|       cost=441 rows=0) (actual time=22.5..22.5 rows=52 loops=1)
|         -> Table scan on t (cost=2.5..2.5 rows=0) (actual time=9.69..9.7 rows=52 loops=1)
|           -> Materialize CTE TotalStars (cost=0..0 rows=0) (actual time=9.69..9.69 rows=52 loops=1)
|             -> Table scan on <temporary> (actual time=9.65..9.65 rows=52 loops=1)
|               -> Aggregate using temporary table (actual time=9.64..9.64 rows=52 loops=1)
|                 -> Filter: ((CelestialObject.Constellation is not null) and (CelestialObject.Constellation <> '')) (cost=3945 rows=2820) (actu
|       al time=0.17..9.51 rows=174 loops=1)
|         -> Index range scan on CelestialObject using idx_magnitude over (NULL < Magnitude < 3), with index condition: (CelestialObj
|       ect.Magnitude < 3) (cost=3945 rows=5639) (actual time=0.161..9.18 rows=5639 loops=1)
|           -> Hash
|             -> Table scan on VisibleStars (cost=2.5..2.5 rows=0) (actual time=11.2..11.2 rows=27 loops=1)
|               -> Materialize CTE VisibleStars (cost=0..0 rows=0) (actual time=11.2..11.2 rows=27 loops=1)
|                 -> Table scan on <temporary> (actual time=11.2..11.2 rows=27 loops=1)
|                   -> Aggregate using temporary table (actual time=11.2..11.2 rows=27 loops=1)
|                     -> Filter: ((CelestialObject.Constellation is not null) and (CelestialObject.Constellation <> '')) and (CelestialObject.Magn
|       itude <= (select #4))) (cost=3856 rows=2756) (actual time=1.3..9.96 rows=50 loops=1)
|                       -> Index range scan on CelestialObject using idx_magnitude over (NULL < Magnitude <= 2), with index condition: (Celesti
|       alObject.Magnitude < 3) (cost=3856 rows=5512) (actual time=1.29..9.63 rows=5512 loops=1)
|                         -> Select #4 (subquery in condition; run only once)
|                           -> Aggregate: avg(ThreeClosest.LimitingMag) (cost=1260..1260 rows=1) (actual time=13.1..13.1 rows=1 loops=1)
|                             -> Table scan on ThreeClosest (cost=1258..1259 rows=3) (actual time=13..13 rows=1 loops=1)
|                               -> Materialize (cost=1257..1257 rows=3) (actual time=13..13 rows=1 loops=1)
|                                 -> Limit: 3 row(s) (cost=1257 rows=3) (actual time=12.3..12.3 rows=1 loops=1)
|                                   -> Sort: abs((40.1164 - LightPollutionObservation.Latitude)), abs((-88.2434) - LightPollutionObser
|       vation.Longitude)), limit input to 3 row(s) per chunk (cost=1257 rows=12489) (actual time=12.3..12.3 rows=1 loops=1)
|                                     -> Filter: ((LightPollutionObservation.Latitude between <cache>{(40.1164 - 0.1448)} and <cache>

```



```
mysql> EXPLAIN ANALYZE SELECT ReportText,
-> CreatedAt,
-> DATEDIFF(NOW(), CreatedAt) AS DaysAgo,
-> COALESCE(AVG(LimitingMag), 6) AS AvgLimitingMag,
-> COUNT(ObservationID) AS NearbyObservations
-> FROM CommunityReport c
-> -- Combine all CommunityReports with any LightPollutionObservations within 10 miles
-> LEFT JOIN LightPollutionObservation l
-> -- 10 land miles = (approx) 0.1448 degrees latitude shift
-> ON l.Latitude BETWEEN c.Latitude - 0.1448 AND c.Latitude + 0.1448
-> -- 10 land miles = longitude shift scaled by latitude for Earth's curvature
-> AND l.Longitude BETWEEN
-> c.Longitude - 10 / 69.17 * COS(RADIANS(c.Latitude))
-> AND c.Longitude + 10 / 69.17 * COS(RADIANS(c.Latitude))
-> -- Filter only CommunityReports within 10 miles of user (assumed to be in Chicago)
-> WHERE c.Latitude BETWEEN 41.8832 - 0.1448 AND 41.8832 + 0.1448
-> AND c.Longitude BETWEEN
-> -87.6324 - 10 / 69.17 * COS(RADIANS(41.8832))
-> AND -87.6324 + 10 / 69.17 * COS(RADIANS(41.8832))
-> GROUP BY ReportID,
-> ReportText,
-> CreatedAt
-> ORDER BY DaysAgo;
```

```

1  -- Champaign: 40.116 x -88.2434
2  -- Chicago: 41.8832 x -87.6324
3  EXPLAIN ANALYZE
4  SELECT ReportText,
5         CreatedAt,
6         DATEDIFF(NOW(), CreatedAt) AS DaysAgo,
7         COALESCE(AVG(LimitingMag), 6) AS AvgLimitingMag,
8         COUNT(ObservationID) AS NearbyObservations
9  FROM CommunityReport c
10     -- Combine all CommunityReports with any LightPollutionObservations within 10 miles
11     LEFT JOIN LightPollutionObservation l
12         -- 10 land miles = (approx) 0.1448 degrees latitude shift
13         ON l.Latitude BETWEEN c.Latitude - 0.1448 AND c.Latitude + 0.1448
14         -- 10 land miles = longitude shift scaled by latitude for Earth's curvature
15         AND l.Longitude BETWEEN
16             c.Longitude - 10 / 69.17 * COS(RADIANS(c.Latitude))
17             AND c.Longitude + 10 / 69.17 * COS(RADIANS(c.Latitude))
18     -- Filter only CommunityReports within 10 miles of user (assumed to be in Chicago)
19 WHERE c.Latitude BETWEEN 41.8832 - 0.1448 AND 41.8832 + 0.1448
20        AND c.Longitude BETWEEN
21            -87.6324 - 10 / 69.17 * COS(RADIANS(41.8832))
22            AND -87.6324 + 10 / 69.17 * COS(RADIANS(41.8832))
23 GROUP BY ReportID,
24          ReportText,
25          CreatedAt
26 ORDER BY DaysAgo;

```

-> Sort: DaysAgo (actual time=79.6..79.6 rows=22 loops=1) -> Table scan on <temporary> (actual time=78.6..78.7 rows=22 loops=1) -> Aggregate using temporary table (actual time=78.6..78.6 rows=22 loops=1) -> Left hash join (no condition), extra conditions: (l.Latitude between (c.Latitude - 0.1448) and (c.Latitude + 0.1448)) and (l.Longitude between (c.Longitude - (<cache>((10 / 69.17)) * cos(radians(c.Latitude)))) and (c.Longitude + (<cache>((10 / 69.17)) * cos(radians(c.Latitude)))) (cost=2513 rows=12489) (actual time=12.5..73.9 rows=63 loops=1) -> Filter: ((c.Latitude between <cache>((41.8832 - 0.1448)) and <cache>((41.8832 + 0.1448))) and (c.Longitude between <cache>((-87.6324) - ((10 / 69.17) * cos(radians(41.8832)))) and <cache>((-87.6324) + ((10 / 69.17) * cos(radians(41.8832))))) (cost=8.35 rows=1) (actual time=3.51..3.99 rows=22 loops=1) -> Table scan on c (cost=8.35 rows=76) (actual time=0.0998..0.709 rows=76 loops=1) -> Hash -> Table scan on l (cost=1257 rows=12489) (actual time=0.783..6.38 rows=12525 loops=1)

Query 1 indexing analysis

Design 1

```
ALTER TABLE CelestialObject
  ADD INDEX idx_constellation(Constellation),
  ADD INDEX idx_magnitude(Magnitude);

EXPLAIN ANALYZE
WITH TotalStars AS (
  -- Table 1: Counts every significant star in each constellation
  SELECT Constellation,
         COUNT(ObjectID) AS StarCount
  FROM CelestialObject
  WHERE Constellation IS NOT NULL
        AND Constellation != ''
        AND Magnitude < 3      -- Only the bright 'connect the dots' stars
  GROUP BY Constellation
),
VisibleStars AS (
  -- Table 2: Only count the stars brighter than the local light pollution
  SELECT Constellation,
         COUNT(ObjectID) AS StarsVisible
  FROM CelestialObject
  WHERE Constellation IS NOT NULL
        AND Constellation != ''
        AND Magnitude < 3      -- Only the bright 'connect the dots' stars
        AND Magnitude <= (
          -- Find the closest local average limiting magnitude
          -- If NULL, defaults to suburban/rural average of 6

```

```
-- Sort: `(VisibleCount / t.StarCount)` DESC, t.StarCount DESC (actual time=24..24 rows=52 loops=1) -> Stream results (cost=441 rows=0) (actual time=23.9..24 rows=52 loops=1) -> Left hash join
(<hash>(VisibleStars.Constellation)=<hash>(t.Constellation)), extra conditions: (VisibleStars.Constellation = t.Constellation) (cost=441 rows=0) (actual time=23.8..23.9 rows=52 loops=1) -> Table scan
on t (cost=2.5..2.5 rows=0) (actual time=10.7..10.7 rows=52 loops=1) -> Materialize CTE TotalStars (cost=0..0 rows=0) (actual time=10.7..10.7 rows=52 loops=1) -> Table scan on <temporary> (actual
time=10.7..10.7 rows=52 loops=1) -> Aggregate using temporary table (actual time=10.7..10.7 rows=52 loops=1) -> Filter: ((CelestialObject.Constellation is not null) and (CelestialObject.Constellation
<> '')) (cost=2538 rows=2820) (actual time=0.119..0.6 rows=174 loops=1) -> Index range scan on CelestialObject using idx_magnitude over (NULL < Magnitude < 3), with index condition:
(CelestialObject.Magnitude < 3) (cost=2538 rows=5639) (actual time=0.118..0.2 rows=5639 loops=1) -> Hash -> Table scan on VisibleStars (cost=2.5..2.5 rows=0) (actual time=13.1..13.1 rows=27
loops=1) -> Materialize CTE VisibleStars (cost=0..0 rows=0) (actual time=13.1..13.1 rows=27 loops=1) -> Table scan on <temporary> (actual time=13..13 rows=27 loops=1) -> Aggregate using
temporary table (actual time=13..13 rows=27 loops=1) -> Filter: ((CelestialObject.Constellation is not null) and (CelestialObject.Constellation <> '')) and (CelestialObject.Magnitude <= (select #4)))
(cost=2481 rows=2756) (actual time=2.57..12.9 rows=50 loops=1) -> Index range scan on CelestialObject using idx_magnitude over (NULL < Magnitude <= 2), with index condition:
(CelestialObject.Magnitude < 3) (cost=2481 rows=5512) (actual time=2.55..12.6 rows=5512 loops=1) -> Select #4 (subquery in condition; run only once) -> Aggregate: avg(ThreeClosest.LimitingMag)
(cost=1263..1263 rows=1) (actual time=142..142 rows=1 loops=1) -> Table scan on ThreeClosest (cost=1261..1263 rows=3) (actual time=142..142 rows=1 loops=1) -> Materialize (cost=1260..1260
rows=3) (actual time=142..142 rows=1 loops=1) -> Limit: 3 row(s) (cost=1260 rows=3) (actual time=142..142 rows=1 loops=1) -> Sort: abs((40.1164 - LightPollutionObservation.Latitude)), abs((-
(88.2434) - LightPollutionObservation.Longitude)), limit input to 3 row(s) per chunk (cost=1260 rows=12489) (actual time=142..142 rows=1 loops=1) -> Filter: ((LightPollutionObservation.Latitude
between <cache>((40.1164 - 0.1448)) and <cache>((40.1164 + 0.1448))) and (LightPollutionObservation.Longitude between <cache>((-88.2434) - ((10 / 69.17) * cos(radians(40.1164)))))) and
<cache>((-88.2434) + ((10 / 69.17) * cos(radians(40.1164)))))) (cost=1260 rows=12489) (actual time=141..142 rows=1 loops=1) -> Table scan on LightPollutionObservation (cost=1260 rows=12489)
(actual time=111..139 rows=12525 loops=1)
```

By indexing on the constellation and magnitude attributes on the CelestialObject table, the performance of the query increased as evidenced by the reduction in the cost, from 17124 to 441.

Design 2

```
ALTER TABLE CelestialObject
  ADD INDEX idx_constellation(Constellation);

ALTER TABLE LightPollutionObservation
  ADD INDEX idx_latitude(Latitude),
  ADD INDEX idx_longitude(Longitude);

EXPLAIN ANALYZE
(
WITH TotalStars AS (
  -- Table 1: Counts every significant star in each constellation
  SELECT Constellation,
         COUNT(ObjectID) AS StarCount
  FROM CelestialObject
  WHERE Constellation IS NOT NULL
         AND Constellation != ''
         AND Magnitude < 3      -- Only the bright 'connect the dots' stars
  GROUP BY Constellation
),
VisibleStars AS (
  -- Table 2: Only count the stars brighter than the local light pollution
  SELECT Constellation,
         COUNT(ObjectID) AS StarsVisible
  FROM CelestialObject
  WHERE Constellation IS NOT NULL
         AND Constellation != ''
         AND Magnitude < 3      -- Only the bright 'connect the dots' stars
         AND Magnitude <= (
```

```
> Sort: (VisibleCount / t.StarCount) DESC, t.StarCount DESC (actual time=1082..1082 rows=52 loops=1) -> Stream results (cost=18864 rows=7921) (actual time=1082..1082 rows=52 loops=1) -> Left
hash join (-hash>(VisibleStars.Constellation)=<hash>(t.Constellation)), extra conditions: (VisibleStars.Constellation = t.Constellation) (cost=18864 rows=7921) (actual time=1082..1082 rows=52
loops=1) -> Table scan on t (cost=14091..14094 rows=89) (actual time=328..328 rows=52 loops=1) -> Materialize CTE TotalStars (cost=14090..14090 rows=89) (actual time=328..328 rows=52
loops=1) -> Group aggregate: count(CelestialObject.ObjectID) (cost=14082 rows=89) (actual time=47.3..327 rows=52 loops=1) -> Filter: ((CelestialObject.Constellation is not null) and
(CelestialObject.Constellation <> '')) and (CelestialObject.Magnitude < 3) (cost=12093 rows=19886) (actual time=32.7..326 rows=174 loops=1) -> Index scan on CelestialObject using idx_constellation
(cost=12093 rows=119325) (actual time=0.164..303 rows=132852 loops=1) -> Hash -> Table scan on VisibleStars (cost=11439..11443 rows=89) (actual time=754..754 rows=27 loops=1) -> Materialize
CTE VisibleStars (cost=11439..11439 rows=89) (actual time=754..754 rows=27 loops=1) -> Group aggregate: count(CelestialObject.ObjectID) (cost=11430 rows=89) (actual time=407..753 rows=27
loops=1) -> Filter: ((CelestialObject.Constellation is not null) and (CelestialObject.Constellation <> '')) and (CelestialObject.Magnitude < 3) and (CelestialObject.Magnitude <= (select #4))) (cost=10767
rows=6628) (actual time=397..752 rows=50 loops=1) -> Index scan on CelestialObject using idx_constellation (cost=10767 rows=119325) (actual time=1.74..726 rows=132852 loops=1) -> Select #4
subquery in condition; run only once -> Aggregate: avg(ThreeClosest.LimitingMag) (cost=10.6..10.6 rows=1) (actual time=3.11..3.11 rows=1 loops=1) -> Table scan on ThreeClosest (cost=8.61..10.3
rows=3) (actual time=3.1..3.1 rows=1 loops=1) -> Materialize (cost=7.76..7.76 rows=3) (actual time=3.1..3.1 rows=1 loops=1) -> Limit: 3 row(s) (cost=7.46 rows=3) (actual time=3.06..3.06 rows=1
loops=1) -> Sort: abs((40.1164 - LightPollutionObservation.Latitude)), abs((-88.2434 - LightPollutionObservation.Longitude)), limit input to 3 row(s) per chunk (cost=7.46 rows=16) (actual
time=3.06..3.06 rows=1 loops=1) -> Filter: (LightPollutionObservation.Latitude between <cache>((40.1164 - 0.1448)) and <cache>((40.1164 + 0.1448))) (cost=7.46 rows=16) (actual time=2.26..2.28
rows=1 loops=1) -> Index range scan on LightPollutionObservation using idx_longitude over (-88.354 <= Longitude <= -88.1328), with index condition: (LightPollutionObservation.Longitude between
<cache>((-88.2434) - ((10 / 69.17) * cos(radians(40.1164)))) and <cache>((-88.2434) + ((10 / 69.17) * cos(radians(40.1164)))))) (cost=7.46 rows=16) (actual time=2.25..2.27 rows=16 loops=1)
```

By indexing the constellation attribute in the celestial object table and the latitude and longitude in the Light Pollution observation table, the cost increased to more than both the original design and the configuration in design 1.

Design 3

```
ALTER TABLE CelestialObject
  ADD INDEX idx_magnitude(Magnitude);

ALTER TABLE LightPollutionObservation
  ADD INDEX idx_latitude(Latitude),
  ADD INDEX idx_longitude(Longitude);

EXPLAIN ANALYZE
(
  WITH TotalStars AS (
    -- Table 1: Counts every significant star in each constellation
    SELECT Constellation,
           COUNT(ObjectID) AS StarCount
    FROM CelestialObject
    WHERE Constellation IS NOT NULL
           AND Constellation != ''
           AND Magnitude < 3      -- Only the bright 'connect the dots' stars
    GROUP BY Constellation
  ),
  VisibleStars AS (
    -- Table 2: Only count the stars brighter than the local light pollution
    SELECT Constellation,
           COUNT(ObjectID) AS StarsVisible
    FROM CelestialObject
    WHERE Constellation IS NOT NULL
           AND Constellation != ''
           AND Magnitude < 3      -- Only the bright 'connect the dots' stars
```

```
-- Sort: '(VisibleCount / t.StarCount)' DESC, t.StarCount DESC (actual time=173..173 rows=52 loops=1) -> Stream results (cost=1088 rows=0) (actual time=173..173 rows=52 loops=1) -> Left hash join
(<hash>(VisibleStars.Constellation)=<hash>(t.Constellation)), extra conditions: (VisibleStars.Constellation = t.Constellation) (cost=1088 rows=0) (actual time=173..173 rows=52 loops=1) -> Table scan
on t (cost=2.5..2.5 rows=0) (actual time=39.3..39.3 rows=52 loops=1) -> Materialize CTE TotalStars (cost=0..0 rows=0) (actual time=39.3..39.3 rows=52 loops=1) -> Table scan on <temporary> (actual
time=39.3..39.3 rows=52 loops=1) -> Aggregate using temporary table (actual time=39.3..39.3 rows=52 loops=1) -> Filter: ((CelestialObject.Constellation is not null) and (CelestialObject.Constellation
<> '')) (cost=2902 rows=4568) (actual time=2.16..39.1 rows=174 loops=1) -> Index range scan on CelestialObject using idx_magnitude over (NULL < Magnitude < 3), with index condition:
(CelestialObject.Magnitude < 3) (cost=2902 rows=5639) (actual time=2.15..38.7 rows=5639 loops=1) -> Hash -> Table scan on VisibleStars (cost=2.5..2.5 rows=0) (actual time=134..134 rows=27
loops=1) -> Materialize CTE VisibleStars (cost=0..0 rows=0) (actual time=134..134 rows=27 loops=1) -> Table scan on <temporary> (actual time=134..134 rows=27 loops=1) -> Aggregate using
temporary table (actual time=134..134 rows=27 loops=1) -> Filter: ((CelestialObject.Constellation is not null) and (CelestialObject.Constellation <> '')) and (CelestialObject.Magnitude <= (select #4)))
(cost=2836 rows=4465) (actual time=30.3..134 rows=50 loops=1) -> Index range scan on CelestialObject using idx_magnitude over (NULL < Magnitude <= 2), with index condition:
(CelestialObject.Magnitude < 3) (cost=2836 rows=5512) (actual time=30.2..133 rows=5512 loops=1) -> Select #4 (subquery in condition; run only once) -> Aggregate: avg(ThreeClosest.LimitingMag)
(cost=10.6..10.6 rows=1) (actual time=2.22..2.22 rows=1 loops=1) -> Table scan on ThreeClosest (cost=8.61..10.3 rows=3) (actual time=2.21..2.21 rows=1 loops=1) -> Materialize (cost=7.76..7.76
rows=3) (actual time=2.21..2.21 rows=1 loops=1) -> Limit: 3 row(s) (cost=7.46 rows=3) (actual time=0.967..0.967 rows=1 loops=1) -> Sort: abs((40.1164 - LightPollutionObservation.Latitude)), abs((-
88.2434 - LightPollutionObservation.Longitude)), limit input to 3 row(s) per chunk (cost=7.46 rows=16) (actual time=0.966..0.966 rows=1 loops=1) -> Filter: (LightPollutionObservation.Latitude
between <cache>((40.1164 - 0.1448)) and <cache>((40.1164 + 0.1448))) (cost=7.46 rows=16) (actual time=0.193..0.213 rows=1 loops=1) -> Index range scan on LightPollutionObservation using
idx_longitude over (-88.354 <= Longitude <= -88.1328), with index condition: (LightPollutionObservation.Longitude between <cache>((-88.2434 - ((10 / 69.17) * cos(radians(40.1164)))))) and <cache>((-
88.2434) + ((10 / 69.17) * cos(radians(40.1164)))))) (cost=7.46 rows=16) (actual time=0.182..0.201 rows=16 loops=1)
```

By indexing the magnitude attribute in the celestial object table and the latitude and longitude in the Light Pollution observation table, we get the second best design based on the costs.

Final index design selected

For Query 1, we selected Design 1, which indexes the Constellation and Magnitude attributes on CelestialObject. This design was selected because it produced by far the lowest estimated cost in the EXPLAIN ANALYZE results. The original query had an estimated cost of about

17,124. Design 2, which indexed Constellation on CelestialObject along with Latitude and Longitude on LightPollutionObservation, actually increased the cost to about 18,864, making it worse than the unindexed original. Design 3, which indexed Magnitude on CelestialObject along with Latitude and Longitude on LightPollutionObservation, performed better at a cost of about 1,088, but still fell short of Design 1. Design 1 performed best because Constellation and Magnitude are the two attributes both CTEs filter and group by directly, so indexing them let MySQL avoid scanning the full CelestialObject table twice, while the added latitude/longitude indexes in Designs 2 and 3 didn't help much since that part of the query already narrows its results with ORDER BY and LIMIT rather than a lookup an index could speed up.

Query 2 indexing analysis

Design 1

```

1 ALTER TABLE CelestialObject
2   ADD INDEX idx_object_cat(ObjectCategory);
3
4 EXPLAIN ANALYZE
5 (
6  SELECT ObjectCategory,
7         COUNT(ObjectID) AS TotalSaved,
8         SUM(ObservedStatus = 'Seen') AS Observed,
9         ROUND(SUM(ObservedStatus = 'Seen') / COUNT(ObjectID) * 100, 0)
10        AS CompletionRate
11  FROM ObservationList
12       NATURAL JOIN SavedObject
13       NATURAL JOIN CelestialObject
14 WHERE ListID = 2 -- provide list id here for query test
15 GROUP BY ObjectCategory
16 HAVING CompletionRate < 100
17 ORDER BY CompletionRate DESC);
18
19 ALTER TABLE CelestialObject
20   DROP INDEX idx_object_cat;
21

```

```

-> Sort: CompletionRate DESC (actual time=1741..1741 rows=9 loops=1) -> Filter: (CompletionRate < 100) (actual time=1740..1740 rows=9 loops=1) -> Table scan on
<temporary> (actual time=1738..1738 rows=19 loops=1) -> Aggregate using temporary table (actual time=1738..1738 rows=19 loops=1) -> Nested loop inner join
(cost=36371 rows=58620) (actual time=0.202..1590 rows=132852 loops=1) -> Index lookup on SavedObject using PRIMARY (ListID=2) (cost=5936 rows=58620) (actual
time=0.173..923 rows=132852 loops=1) -> Single-row index lookup on CelestialObject using PRIMARY (ObjectID=SavedObject.ObjectID) (cost=0.419 rows=1) (actual
time=0.00477..0.0048 rows=1 loops=132852)

```

Based on the above, we see that the cost has increased in comparison to the original design without an additional index (to the primary key). Hence the performance has worsened

Design 2

```
ALTER TABLE SavedObject
  ADD INDEX idx_observation_status(ObservedStatus);

EXPLAIN ANALYZE
(
SELECT ObjectCategory,
       COUNT(ObjectID) AS TotalSaved,
       SUM(ObservedStatus = 'Seen') AS Observed,
       ROUND(SUM(ObservedStatus = 'Seen') / COUNT(ObjectID) * 100, 0)
           AS CompletionRate
FROM ObservationList
   NATURAL JOIN SavedObject
   NATURAL JOIN CelestialObject
WHERE ListID = 2 -- provide list id here for query test
GROUP BY ObjectCategory
HAVING CompletionRate < 100
ORDER BY CompletionRate DESC);

ALTER TABLE SavedObject
  DROP INDEX idx_observation_status;
```

```
-> Sort: CompletionRate DESC (actual time=1424..1424 rows=9 loops=1) -> Filter: (CompletionRate < 100) (actual time=1422..1422 rows=9 loops=1) -> Table scan on
<temporary> (actual time=1419..1419 rows=19 loops=1) -> Aggregate using temporary table (actual time=1419..1419 rows=19 loops=1) -> Nested loop inner join
(cost=47476 rows=58620) (actual time=30.9..1285 rows=132852 loops=1) -> Index lookup on SavedObject using PRIMARY (ListID=2) (cost=5897 rows=58620) (actual
time=0.929..365 rows=132852 loops=1) -> Single-row index lookup on CelestialObject using PRIMARY (ObjectID=SavedObject.ObjectID) (cost=0.609 rows=1) (actual
time=0.00667..0.0067 rows=1 loops=132852)
```

Based on the above we can observe that indexing observation status from the saved object results in the highest cost so far hence its worsening performance.

Design 3

```
ALTER TABLE SavedObject
  ADD INDEX idx_observation_status(ObservedStatus);

ALTER TABLE CelestialObject
  ADD INDEX idx_object_cat(ObjectCategory);

EXPLAIN ANALYZE
(
SELECT ObjectCategory,
       COUNT(ObjectID) AS TotalSaved,
       SUM(ObservedStatus = 'Seen') AS Observed,
       ROUND(SUM(ObservedStatus = 'Seen') / COUNT(ObjectID) * 100, 0)
           AS CompletionRate
FROM ObservationList
   NATURAL JOIN SavedObject
   NATURAL JOIN CelestialObject
WHERE ListID = 2                                -- provide list id here for query test
GROUP BY ObjectCategory
HAVING CompletionRate < 100
ORDER BY CompletionRate DESC);

ALTER TABLE SavedObject
  DROP INDEX idx_observation_status;

ALTER TABLE CelestialObject
  DROP INDEX idx_object_cat;
```

```
-> Sort: CompletionRate DESC (actual time=1146..1146 rows=9 loops=1) -> Filter: (CompletionRate < 100) (actual time=1145..1145 rows=9 loops=1) -> Table scan on
<temporary> (actual time=1145..1145 rows=19 loops=1) -> Aggregate using temporary table (actual time=1145..1145 rows=19 loops=1) -> Nested loop inner join
(cost=28386 rows=58620) (actual time=0.116..1011 rows=132852 loops=1) -> Index lookup on SavedObject using PRIMARY (ListID=2) (cost=5926 rows=58620) (actual
time=0.0874..556 rows=132852 loops=1) -> Single-row index lookup on CelestialObject using PRIMARY (ObjectID=SavedObject.ObjectID) (cost=0.283 rows=1) (actual
time=0.0032..0.00323 rows=1 loops=132852)
```

Design 3 results in lower costs than design 1 and 2 but not yet below the original cost before any indexing at all.

Final index design selected

Based on this, we choose to retain the original design - without any indexing. This is because it has the lowest cost and hence we expect a better performance in its original form. One possible explanation as to why indexing more attributes seems to be less effective at reducing the cost might be because there are relatively fewer attributes in the where, join and groupby clauses to use for indexing. Since indexing attributes in those clauses seems to be more likely to improve

performance, if a query has fewer attributes in those clauses it makes sense if further indexing doesn't necessarily improve performance. The other potential factor is that query 2 is a shorter and more precise query than other queries and hence it would naturally require less indexing or no indexing at all for improved performance.

Query 3 indexing analysis

Design 1

```

1 CREATE INDEX idx_community_lat_long
2   ON CommunityReport(Latitude, Longitude);
3
4 -- Champaign: 40.116 x -88.2434
5 -- Chicago: 41.8832 x -87.6324
6 EXPLAIN ANALYZE
7 SELECT ReportText,
8        CreatedAt,
9        DATEDIFF(NOW(), CreatedAt) AS DaysAgo,
10       COALESCE(AVG(LimitingMag), 6) AS AvgLimitingMag,
11       COUNT(ObservationID) AS NearbyObservations
12 FROM CommunityReport c
13      -- Combine all CommunityReports with any LightPollutionObservations within 10 miles
14      LEFT JOIN LightPollutionObservation l
15      -- 10 land miles = (approx) 0.1448 degrees latitude shift
16      ON l.Latitude BETWEEN c.Latitude - 0.1448 AND c.Latitude + 0.1448
17      -- 10 land miles = longitude shift scaled by latitude for Earth's curvature
18      AND l.Longitude BETWEEN
19      | c.Longitude - 10 / 69.17 * COS(RADIANS(c.Latitude))
20      | AND c.Longitude + 10 / 69.17 * COS(RADIANS(c.Latitude))
21 -- Filter only CommunityReports within 10 miles of user (assumed to be in Chicago)
22 WHERE c.Latitude BETWEEN 41.8832 - 0.1448 AND 41.8832 + 0.1448
23      AND c.Longitude BETWEEN
24      | -87.6324 - 10 / 69.17 * COS(RADIANS(41.8832))
25      | AND -87.6324 + 10 / 69.17 * COS(RADIANS(41.8832))
26 GROUP BY ReportID,
27        ReportText,
28        CreatedAt
29 ORDER BY DaysAgo;
30

```

-> Sort: DaysAgo (actual time=93.2..93.2 rows=22 loops=1) -> Table scan on <temporary> (actual time=93..93 rows=22 loops=1) -> Aggregate using temporary table (actual time=93..93 rows=22 loops=1)
-> Left hash join (no condition), extra conditions: ((l.Latitude between (c.Latitude - 0.1448) and (c.Latitude + 0.1448)) and (l.Longitude between (c.Longitude - (<cache>((10 / 69.17)) * cos(radians(c.Latitude)))) and (c.Longitude + (<cache>((10 / 69.17)) * cos(radians(c.Latitude)))))) (cost=31683 rows=312225) (actual time=11.5..91.7 rows=63 loops=1) -> Index range scan on c using idx_community_lat_long over (41.7384 <= Latitude <= 42.028 AND Longitude < -87.5248), with index condition: ((c.Latitude between <cache> ((41.8832 - 0.1448)) and <cache>((41.8832 + 0.1448))) and (c.Longitude between <cache>((-87.6324) - ((10 / 69.17) * cos(radians(41.8832)))) and <cache>((-87.6324) + ((10 / 69.17) * cos(radians(41.8832)))))) (cost=11.5 rows=25) (actual time=0.967..1.58 rows=22 loops=1) -> Hash -> Table scan on l (cost=452 rows=12489) (actual time=0.534..6.86 rows=12525 loops=1)

Design 1 adds an index on CommunityReport(Latitude, Longitude) to help filter community reports near Chicago. MySQL did use this index with an index range scan, but the main estimated cost increased from about 2513 to 31683. So even though the index was used, it did not improve the overall query performance.

Design 2

```

1 CREATE INDEX idx_lightpollution_lat_long
2   ON LightPollutionObservation(Latitude, Longitude);
3 -- Champaign: 40.116 x -88.2434
4 -- Chicago: 41.8832 x -87.6324
5
6 EXPLAIN ANALYZE
7 SELECT ReportText,
8        CreatedAt,
9        DATEDIFF(NOW(), CreatedAt) AS DaysAgo,
10       COALESCE(AVG(LimitingMag), 6) AS AvgLimitingMag,
11       COUNT(ObservationID) AS NearbyObservations
12 FROM CommunityReport c
13      -- Combine all CommunityReports with any LightPollutionObservations within 10 miles
14 LEFT JOIN LightPollutionObservation l
15      -- 10 land miles = (approx) 0.1448 degrees latitude shift
16   ON l.Latitude BETWEEN c.Latitude - 0.1448 AND c.Latitude + 0.1448
17      -- 10 land miles = longitude shift scaled by latitude for Earth's curvature
18   AND l.Longitude BETWEEN
19      c.Longitude - 10 / 69.17 * COS(RADIANS(c.Latitude))
20      AND c.Longitude + 10 / 69.17 * COS(RADIANS(c.Latitude))
21 -- Filter only CommunityReports within 10 miles of user (assumed to be in Chicago)
22 WHERE c.Latitude BETWEEN 41.8832 - 0.1448 AND 41.8832 + 0.1448
23      AND c.Longitude BETWEEN
24      -87.6324 - 10 / 69.17 * COS(RADIANS(41.8832))
25      AND -87.6324 + 10 / 69.17 * COS(RADIANS(41.8832))
26 GROUP BY ReportID,
27          ReportText,
28          CreatedAt
29 ORDER BY DaysAgo;

```

```

-> Sort: DaysAgo (actual time=172..172 rows=22 loops=1) -> Table scan on <temporary> (actual
time=172..172 rows=22 loops=1) -> Aggregate using temporary table (actual time=172..172 rows=22
loops=1) -> Nested loop left join (cost=1265 rows=12489) (actual time=4.77..168 rows=63 loops=1) ->
Filter: ((c.Latitude between <cache>((41.8832 - 0.1448)) and <cache>((41.8832 + 0.1448))) and
(c.Longitude between <cache>((-87.6324) - ((10 / 69.17) * cos(radians(41.8832)))) and <cache>((-
87.6324) + ((10 / 69.17) * cos(radians(41.8832)))))) (cost=8.35 rows=1) (actual time=0.161..0.4
rows=22 loops=1) -> Table scan on c (cost=8.35 rows=76) (actual time=0.0395..0.29 rows=76 loops=1) ->
Filter: ((l.Latitude between (c.Latitude - 0.1448) and (c.Latitude + 0.1448)) and (l.Longitude between
(c.Longitude - (<cache>((10 / 69.17) * cos(radians(c.Latitude)))) and (c.Longitude + (<cache>((10 /
69.17) * cos(radians(c.Latitude)))))) (cost=1257 rows=12489) (actual time=2.85..7.58 rows=2.86
loops=22) -> Index range scan on l (re-planned for each iteration) (cost=1257 rows=12489) (actual
time=0.304..4.78 rows=12525 loops=22)

```

Design 2 adds an index on LightPollutionObservation(Latitude, Longitude) to help find nearby light pollution observations during the join. This was the best design because it lowered the main estimated cost from about 2513 to 1265. Since the query mainly depends on matching nearby location data, this index was the most useful.

Design 3

```
1 CREATE INDEX idx_community_lat_long_created
2   ON CommunityReport(Latitude, Longitude, CreatedAt);
3 -- Champaign: 40.116 x -88.2434
4 -- Chicago: 41.8832 x -87.6324
5
6 EXPLAIN ANALYZE
7 SELECT ReportText,
8        CreatedAt,
9        DATEDIFF(NOW(), CreatedAt) AS DaysAgo,
10       COALESCE(AVG(LimitingMag), 6) AS AvgLimitingMag,
11       COUNT(ObservationID) AS NearbyObservations
12 FROM CommunityReport c
13      -- Combine all CommunityReports with any LightPollutionObservations within 10 miles
14      LEFT JOIN LightPollutionObservation l
15      ON 10 land miles = (approx) 0.1448 degrees latitude shift
16      ON l.Latitude BETWEEN c.Latitude - 0.1448 AND c.Latitude + 0.1448
17      -- 10 land miles = longitude shift scaled by latitude for Earth's curvature
18      AND l.Longitude BETWEEN
19          c.Longitude - 10 / 69.17 * COS(RADIANS(c.Latitude))
20          AND c.Longitude + 10 / 69.17 * COS(RADIANS(c.Latitude))
21 -- Filter only CommunityReports within 10 miles of user (assumed to be in Chicago)
22 WHERE c.Latitude BETWEEN 41.8832 - 0.1448 AND 41.8832 + 0.1448
23      AND c.Longitude BETWEEN
24          -87.6324 - 10 / 69.17 * COS(RADIANS(41.8832))
25          AND -87.6324 + 10 / 69.17 * COS(RADIANS(41.8832))
26 GROUP BY ReportID,
27        ReportText,
28        CreatedAt
29 ORDER BY DaysAgo;
```

```

-> Sort: DaysAgo (actual time=74..74 rows=22 loops=1) -> Table scan on <temporary> (actual
time=73.9..73.9 rows=22 loops=1) -> Aggregate using temporary table (actual time=73.9..73.9 rows=22
loops=1) -> Left hash join (no condition), extra conditions: (l.Latitude between (c.Latitude - 0.1448) and
(c.Latitude + 0.1448)) and (l.Longitude between (c.Longitude - (<cache>((10 / 69.17)) *
cos(radians(c.Latitude)))) and (c.Longitude + (<cache>((10 / 69.17)) * cos(radians(c.Latitude))))
(cost=31683 rows=312225) (actual time=10.4..72.4 rows=63 loops=1) -> Index range scan on c using
idx_community_lat_long_created over (41.7384 <= Latitude <= 42.028 AND Longitude < -87.5248), with
index condition: ((c.Latitude between <cache>((41.8832 - 0.1448)) and <cache>((41.8832 + 0.1448))) and
(c.Longitude between <cache>((-87.6324) - ((10 / 69.17) * cos(radians(41.8832)))) and <cache>((-
87.6324) + ((10 / 69.17) * cos(radians(41.8832))))) (cost=11.5 rows=25) (actual time=0.0636..0.632
rows=22 loops=1) -> Hash -> Table scan on l (cost=452 rows=12489) (actual time=0.409..4.29
rows=12525 loops=1)

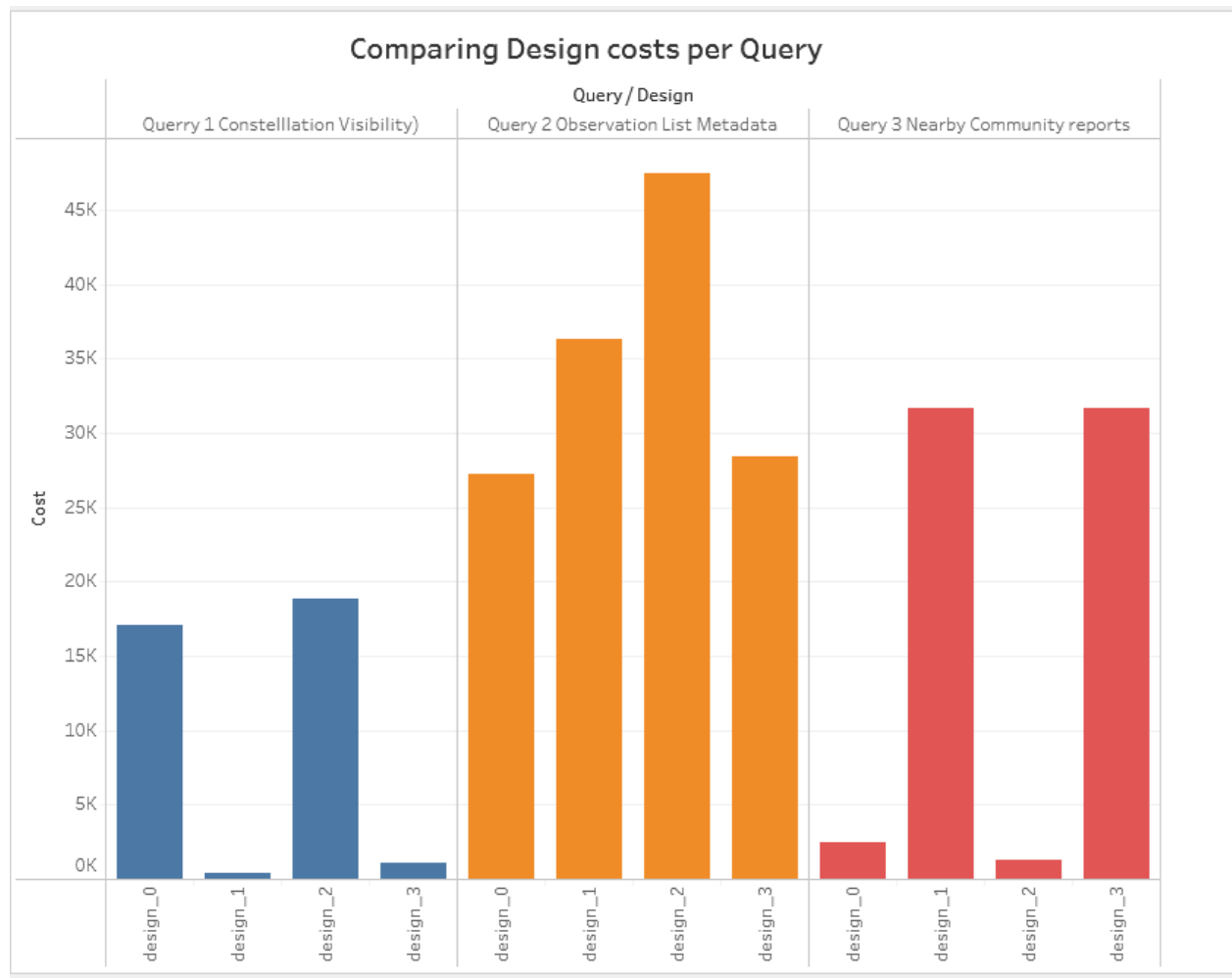
```

Design 3 adds an index on CommunityReport(Latitude, Longitude, CreatedAt) to help with location filtering and date ordering. MySQL used the index, but the main estimated cost was still about 31683. This means adding CreatedAt did not help much because the expensive part of the query is the location-based join, not the ordering.

Final index design selected

For Query 3, we selected Design 2, which creates an index on the latitude and longitude attributes of LightPollutionObservation. This design was selected because it produced the lowest estimated cost in the EXPLAIN ANALYZE results. The original query had an estimated main join cost of about 2513. Design 1, which indexed CommunityReport(Latitude, Longitude), changed the plan to use an index range scan on CommunityReport, but the main cost increased to about 31683. Design 3, which indexed CommunityReport(Latitude, Longitude, CreatedAt), also used an index range scan on CommunityReport, but its main cost was still about 31683. Design 2 performed best because it lowered the estimated cost to about 1265 by indexing LightPollutionObservation(Latitude, Longitude), which directly supports the location based join condition used to find nearby light pollution observations for each community report.

Design comparisons graph for each Query



The design_0 columns refer to the initial cost of each query prior to indexing.